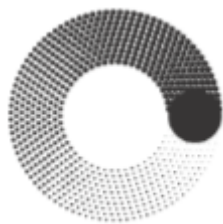


**федеральное государственное автономное образовательное
учреждение высшего образования**



**МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(ВЫСШАЯ ШКОЛА ПЕЧАТИ И МЕДИАИНДУСТРИИ)
(Факультет информационных технологий)**

*(Институт Принтмедиа и информационных технологий)
Кафедра Информатики и информационных технологий*

направление подготовки

09.03.02 «Информационные системы и технологии»

ЛАБОРАТОРНАЯ РАБОТА № 5

Дисциплина: BackEnd-разработка _____

Тема: «Добавление маршрутизации в веб-приложение на основе ASP.NET Core
»

Выполнил(а): студент(ка) группы 241-333

Конев М. Д.
(Фамилия И.О.)

Дата, подпись 15.05.2026
(Дата)


(Подпись)

Проверил: _____
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись _____
(Дата) (Подпись)

Замечания: _____

Москва 2026

Цель занятия: познакомиться с основами настройки и использования маршрутизации в веб-приложениях на основе ASP.NET Core

Задания:

1. Расширить существующее приложение из лабораторной №3, добавив новые маршруты для нескольких ресурсов (например, products, students, orders).
2. Настроить маршрутизацию для контроллеров с базовым маршрутом api/[controller].
3. Реализовать маршруты для действий контроллера:

GET api/<controller>

GET api/<controller>/{id}

POST api/<controller>

PUT api/<controller>/{id}

DELETE api/<controller>/{id}

4. Создать маршруты с параметрами: {id}, {name}, {year} и т.п.
5. Добавить ограничения маршрутов:

{id:int}

{date:datetime}

{guid:guid}

{slug:minlength(3)}

6. Реализовать необязательные параметры в маршрутах: {id?} и значения по умолчанию (где применимо).
7. Реализовать маршрут с вложенной структурой (например, api/students/{id}/courses).
8. Добавить маршруты с query-параметрами (например, фильтрация/сортировка): ?page=1&pageSize=10&sort=name.
9. Протестировать маршрутизацию через Swagger и/или Postman: проверить попадание в нужный метод, статус-коды и ответы.
10. Зафиксировать результаты тестирования: список URL -> какой метод вызывается -> ожидаемый результат

Ход выполнения работы:

1. Структура проекта

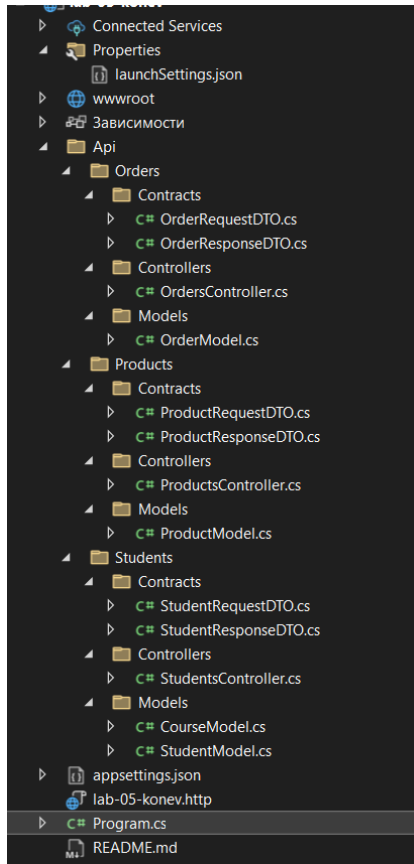


Рисунок 1 структура проекта

Такое разделение обеспечивает удобную организацию кода и облегчает поддержку проекта.

2. api/[controller]

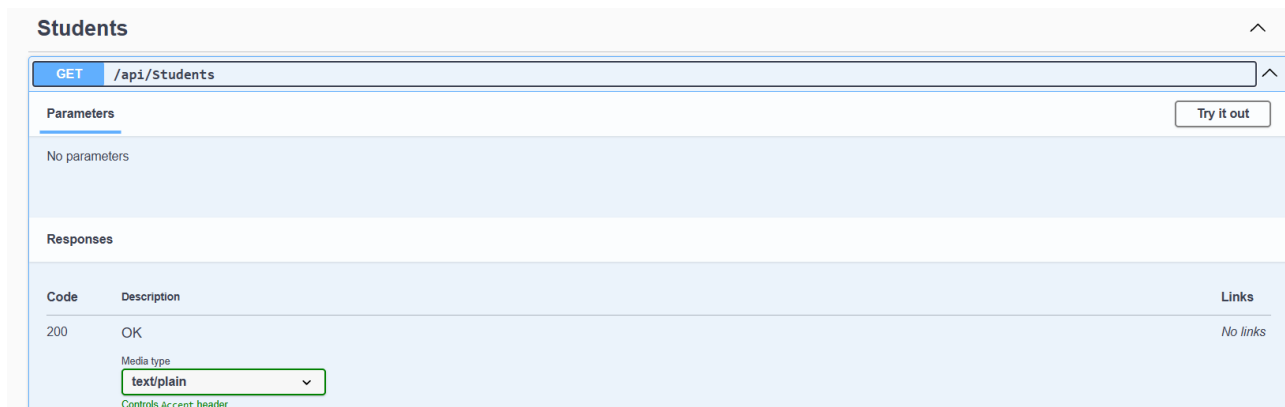
```
// Базовый маршрут контроллера: api/orders
[Route("api/[controller]")]
[ApiController]
```

Рисунок 2 базовая маршрутизация

Базовая маршрутизация api

3. GET списка

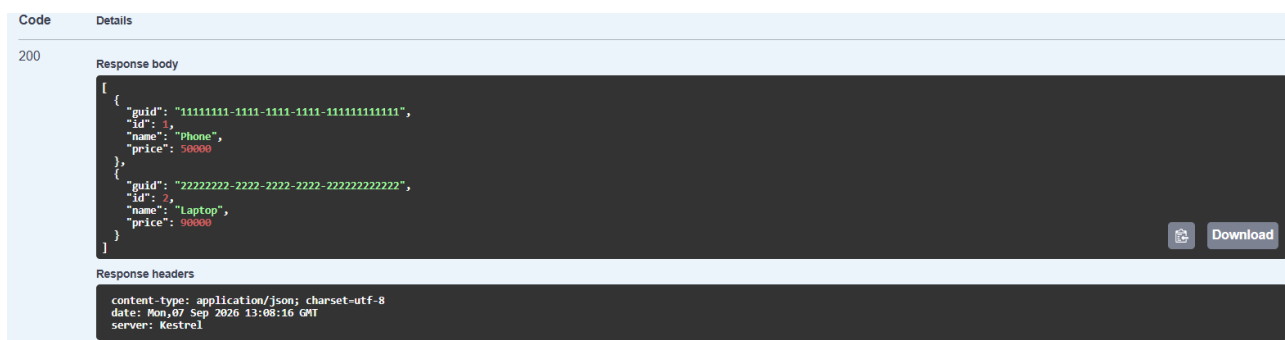
```
public ActionResult<StudentResponseDTO[]> GetAllStudents()
{
    ...
    return Ok(_students.Select(ToResponse).ToArray());
}
```



Метод GET возвращает список объектов выбранного ресурса. При успешном выполнении сервер возвращает статус 200 OK и данные в формате JSON.

4. GET {id:int}

```
Ссылка: 0
public ActionResult<ProductResponseDTO[]> GetProducts()
{
    return Ok(_products.Select(ToResponse).ToArray());
}
```



Маршрут позволяет получить объект по целочисленному идентификатору. Если объект не найден, сервер возвращает 404 Not Found.

5. POST

```
public ActionResult<OrderResponseDTO[]> GetOrders()
{
    return Ok(_orders.Select(ToResponse).ToArray());
}
```

Server response	
Code	Details
201 <i>Undocumented</i>	Response body <pre>{ "id": 4, "title": "string", "date": "07.09.2026 13:11", "price": 1000000 }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Mon, 07 Sep 2026 13:11:59 GMT location: https://localhost:7087/api/Orders/4 server: Kestrel</pre>

Метод POST создаёт новый объект на основе данных из тела запроса. При успешном создании возвращается статус 201 Created.

6. PUT {id:int}

```
// PUT api/students/1
[HttpPut("{id:int}")]
Ссылка: 0
public ActionResult UpdateStudent(int id, [FromBody] StudentRequestDTO contract)
{
    var student = _students.FirstOrDefault(x => x.Id == id);
    if (student is null)
        return NotFound("Студент не найден.");

    student.Name = contract.Name;
    student.Group = contract.Group;
    student.Specialization = contract.Specialization;
    student.Courses = contract.Courses;

    return NoContent();
}
```

Server response	
Code	Details
204 <i>Undocumented</i>	Response headers <pre>date: Mon, 07 Sep 2026 13:14:06 GMT server: Kestrel</pre> Responses

Метод PUT обновляет данные существующего объекта по указанному идентификатору. После успешного обновления сервер возвращает 204 No Content.

7. DELETE {id:int}

```
// DELETE api/orders/1
[HttpDelete("{id:int}")]
Ссылка: 0
public ActionResult DeleteOrder(int id)
{
    var index = _orders.FindIndex(x => x.Id == id);
    if (index < 0)
        return NotFound("Заказ не найден.");

    _orders.RemoveAt(index);
    return NoContent();
}
```

Server response	
Code	Details
204	Response headers
Undocumented	date: Mon, 07 Sep 2020 13:17:10 GMT server: Kestrel

Метод DELETE удаляет объект по идентификатору. При успешном удалении возвращается статус 204 No Content.

8. {year:int}

```
// GET api/orders/year/2024
// Параметр {year:int} принимает только целое число.
[HttpGet("year/{year:int}")]
Ссылка: 0
public ActionResult<OrderResponseDTO[]> GetOrdersByYear(int year)
{
    var orders = _orders
        .Where(x => x.Date.Year == year)
        .Select(ToResponse)
        .ToArray();

    return orders.Length > 0
        ? Ok(orders)
        : NotFound("Заказы за этот год не найдены.");
}
```

Server response	
Code	Details
200	Response body
	<pre>[{ "id": 1, "title": "Order Alpha", "date": "10.03.2024 12:00", "price": 1200 }]</pre>

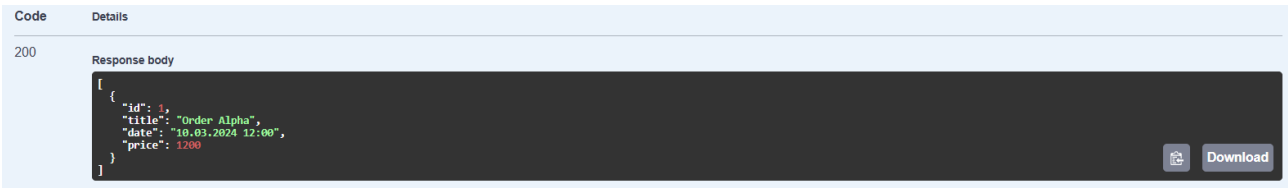
Маршрут с параметром year позволяет получить заказы за указанный год.

Ограничение int требует передачи целого числа.

9. {date:datetime}

```
// GET api/orders/date/2024-03-10
// Ограничение {date:datetime}: значение должно быть корректной датой.
[HttpGet("date/{date:datetime}")]
Ссылка: 0
public ActionResult<OrderResponseDTO[]> GetOrdersByDate(DateTime date)
{
    var orders = _orders
        .Where(x => x.Date.Date == date.Date)
        .Select(ToResponse)
        .ToArray();

    return orders.Length > 0
        ? Ok(orders)
        : NotFound("Заказы на эту дату не найдены.");
}
```



The screenshot shows a REST client interface with a 'Code' tab selected. The status code is 200. The 'Response body' tab shows a JSON array containing one object: `[{"id": 1, "title": "Order Alpha", "date": "10.03.2024 12:00", "price": 1200}]`. A 'Download' button is visible at the bottom right of the response area.

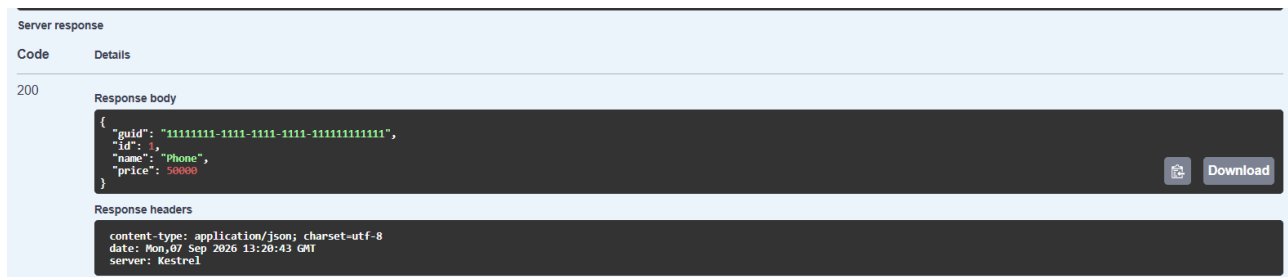
Маршрут с параметром date используется для поиска заказов по дате.

Ограничение datetime проверяет корректность формата переданного значения.

10. {guid:guid}

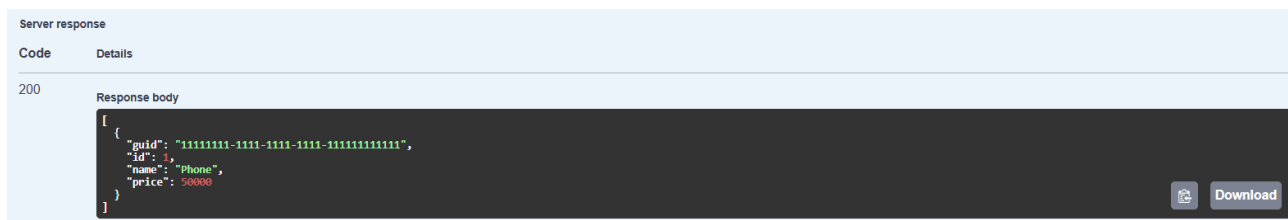
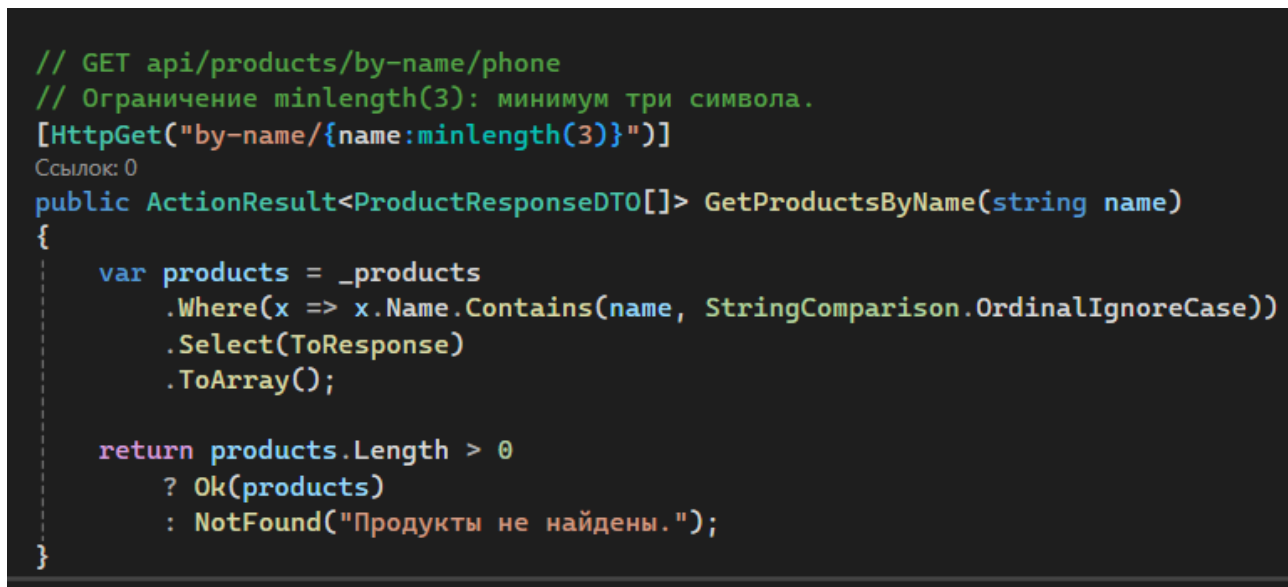
```
// GET api/products/{guid}
// Ограничение {guid:guid}: только корректный GUID.
[HttpGet("{guid:guid}")]
Ссылка: 0
public ActionResult<ProductResponseDTO> GetProductByGuid(Guid guid)
{
    var product = _products.FirstOrDefault(x => x.Guid == guid);
    if (product is null)
        return NotFound("Продукт не найден.");

    return Ok(ToResponse(product));
}
```



Маршрут позволяет получить продукт по уникальному идентификатору GUID. Ограничение `guid` обеспечивает соответствие параметра необходимому формату.

11. `{name:minlength(3)}`



Маршрут выполняет поиск продуктов по имени. Ограничение `minlength(3)` требует, чтобы параметр содержал не менее трёх символов.

12. Вложенный {id}/courses

```
// Вложенный маршрут: GET api/students/1/courses
[HttpGet("{id:int}/courses")]
Ссылки: 0
public ActionResult<List<Course>> GetStudentCourses(int id)
{
    var student = _students.FirstOrDefault(x => x.Id == id);
    if (student is null)
        return NotFound("Студент не найден.");

    return Ok(student.Courses);
}
```

Curl

```
curl -X 'GET' \
  'https://localhost:7087/api/Students/1/courses' \
  -H 'accept: text/plain'
```

Request URL

https://localhost:7087/api/Students/1/courses

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "title": "ASP.NET Core" }, { "id": 2, "title": "Databases" }]</pre> Download

Вложенный маршрут позволяет получить список курсов конкретного студента. Идентификатор студента передается в URL.

13. Query ?group=

```
// Query-параметр
// GET api/students/group?group=2
[HttpGet("group")]
Ссылки: 0
public ActionResult<StudentResponseDTO[]> GetStudentsByGroup([FromQuery] int? group = null)
{
    var students = _students
        .Where(x => group is null || x.Group == group.Value)
        .Select(ToResponse)
        .ToArray();

    return students.Length > 0
        ? Ok(students)
        : NotFound("Студенты не найдены.");
}
```



Query-параметр `group` используется для фильтрации студентов по учебной группе. Если параметр не указан, возвращаются все студенты.

14. Необязательный `{id?}`

Необязательный параметр позволяет выполнять запрос как с идентификатором, так и без него. При отсутствии `id` возвращается список студентов, а при его указании — конкретный объект.

15. `?page=1&pageSize=10&sort=name`

Query-параметры `page` и `pageSize` задают постраничное получение данных, а `sort` определяет порядок сортировки. Это позволяет управлять выборкой без изменения базового маршрута.

16. `{slug:minlength(3)}`

Маршрут использует текстовый идентификатор `slug` для поиска ресурса.

Ограничение `minlength(3)` требует длину параметра не менее трёх символов.

17. Значение по умолчанию

В маршруте реализовано значение параметра по умолчанию. Если параметр не передан в URL, приложение использует заранее заданное значение.